KCC CODE CLUB · GAME MAKERS

Week 6 Lesson Plan

Broadcasts and Game Over

Big idea: Sprites can't see each other's code, but they can shout a message that everyone hears. That's how the different parts of a program work together.

At a glance

New concepts	Message passing, program state, coordinating multiple sprites
New blocks	`broadcast`, `broadcast and wait`, `when I receive`, `switch backdrop to`, `stop all`, `stop this script`, `show`, `hide`
Class activity	Magic Wand
Project checkpoint	Lives, a title screen, a game over screen, and a restart
Homework	A win condition; a restart

Before the class

- Print the Week 6 handout.
- Skeleton `Sky Catcher — end of Week 5` ready.
- This class has the most moving parts of any in the course. Read it twice.

0:00–0:05 — Show & Tell

Ask for high scores specifically. Get two up on the projector and let the class call out whether they could beat them. Keep it to five minutes — today's build is the biggest of the term and it needs every one of its twenty-five.

0:05–0:15 — Live demo: broadcasts and backdrops

Build live with two sprites:

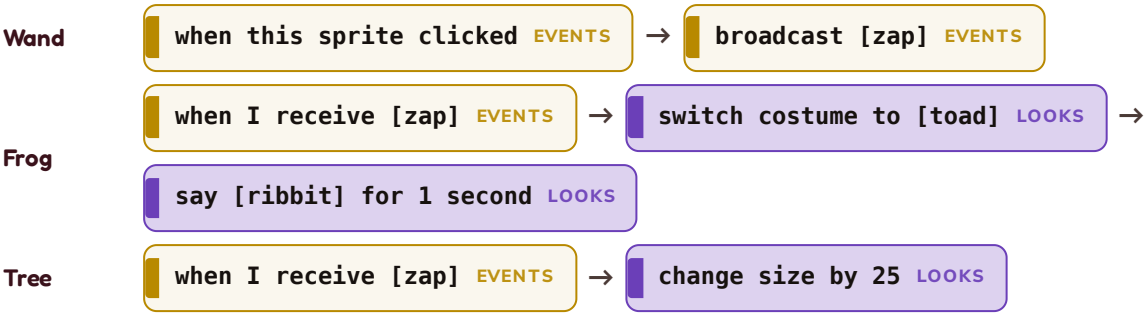
1. On sprite A: `when this sprite clicked` → `broadcast [transform]`.
2. On sprite B: `when I receive [transform]` → `switch costume to [something]` → `change size by 50`.
3. Click sprite A. Sprite B reacts. Emphasise that sprite A has no idea sprite B exists.
4. Add a **third** sprite that also listens for `transform` and does something different. One shout, three reactions.
5. Show `switch backdrop to [Game Over]` inside a `when I receive`.
6. Show `stop all` and what it does. Then `stop this script` and the difference.

7. **Deliberate mistake:** broadcast a message that nothing is listening for (typo in the message name, or two similarly-named messages). Nothing happens. Be confused, then find it in the dropdown.

That typo bug is extremely common today because message names are typed by hand. Warn them.

0:15–0:30 — Class activity: Magic Wand

A wand sprite and two or three victim sprites.



Add a second message, `undo`, on a different key, that puts everything back. Making both directions work is what forces them to think about state. If the room is slow, skip `undo` and move on at the quarter-hour; one message, three reactions, is the concept.

0:30–0:55 — Project build: lives, game over, and a start screen

This is the biggest build of the term. Plan on lives and game over for everyone. The title screen is for whoever gets there, and homework for the rest.

Next week's level script starts on `when I receive [startgame]`, which only exists once the title screen does. Anyone who didn't get to it uses `when green flag clicked` instead, or picks up the `end of Week 6` skeleton.

Step 1 — Lives

1. Make a variable `lives`. `set lives to 3` under the green flag.
2. In the bad object's catch `if`, add `change lives by -1`.
3. On the player (or stage):



Step 2 — Game over

1. Add a "Game Over" backdrop (paint one with text, or use a plain one and a text sprite).

2. On the stage: `when I receive [gameover]` → `switch backdrop to [Game Over]` → `stop all`.
3. On each falling object: `when I receive [gameover]` → `hide`.

Note the ordering trap: `stop all` stops *everything*, including the scripts that were about to hide the sprites. Either hide first and stop last, or use a `wait 0.1` before the `stop all`. This is a lovely real bug and worth letting them find.

Step 3 — Title screen (if time, otherwise homework)

1. A "Title" backdrop with the game's name.
2. `when green flag clicked` → `switch backdrop to [Title]` → `say [Press S to start]`.
3. `when [s] key pressed` → `broadcast [startgame]`.
4. Move everything that was under `when green flag clicked` to `when I receive [startgame]`.

That last move is conceptually significant and they'll feel it: the green flag no longer starts the game, it starts the *title screen*. The game has states now.

Save.

0:55–1:00 — Wrap

Save and back up. Today's project has more scripts spread across more sprites than any previous week, and a lost file now costs real work.

Homework

1. **A way to win.** Pick a target score and broadcast a `youwin` message when the player reaches it, with its own backdrop.
2. **Restart.** Add a way to play again without clicking the green flag — a key press or a clickable button that resets the variables and broadcasts `startgame`.

Bonus challenges

1. Add a "3 — 2 — 1 — GO!" countdown between the title screen and the game starting.
2. **Two-Scene Story.** Two backdrops, two sprites, a story that runs itself. The stage switches to `scene1` and broadcasts `scene1start`; a sprite hears it, says its line, and broadcasts `scene2start`; the stage hears that and switches to `scene2`. Keep going for a third scene.
3. Add a boss object that appears only when the score passes 20.
4. Try `broadcast and wait` instead of `broadcast` somewhere and work out what's different. (Hint: it doesn't move on until every receiver has finished.)

Common bugs this week

Symptom	Cause
Broadcast does nothing	Message name typo, or two near-identical messages in the dropdown

Symptom	Cause
Sprites stay visible after game over	`stop all` ran before the `hide` scripts got a chance. Add a short wait, or hide before stopping
Sprites are invisible when the game restarts	They were hidden at game over and never `show`n again. Add `show` under the start
Lives goes to −1, −2, −3	The `if lives = 0` check missed the exact moment. Use `< 1` instead of `= 0`
Game over fires immediately	`lives` wasn't set to 3 before the check started running
Everything stopped and won't restart	`stop all` really does stop everything, including restart scripts. Restart needs to be on a `when key pressed` hat, which still fires

The `= 0` versus `< 1` one is genuinely instructive. Exact equality checks are fragile because they only catch one specific instant. It's a real lesson in defensive programming, delivered at exactly the right moment.

Notes for the second volunteer

Today's debugging is different in kind from previous weeks: the code is *correct* in each sprite but the *interaction* between sprites is wrong. Teach them to trace the message — "who shouts this, and who's listening?" — rather than staring at one script. Getting a child to draw the messages on paper with arrows works remarkably well.

Links

- Broadcasting spells — Raspberry Pi Foundation, almost exactly the Magic Wand activity with more depth
- Skeleton: `Sky Catcher — end of Week 6` in the club studio